Project Report

Threat Intelligence Aggregator (Non-AI)

Student Name: Harshada Arun Rajale

Course: Unified Mentor - Cybersecurity Intern

Date: 03/04/2026

---

1. Executive Summary

During the course of this internship module, a custom Threat Intelligence (TI) Aggregator was engineered to address the critical challenge of data correlation within a Security Operations Center (SOC). The objective was to build an automated, frictionless tool capable of ingesting external Open-Source Intelligence (OSINT) watchlists and cross-referencing them against internal network firewall logs. The resulting platform was successfully deployed as a client-side web application, demonstrating high-speed data parsing, algorithmic correlation, and the automated generation of actionable master blocklists.

2. Problem Statement & Industry Context

Modern SOC analysts suffer from "alert fatigue" due to the overwhelming volume of unstructured threat data. When a new OSINT watchlist is published, analysts must manually verify if any of those malicious IP addresses, domains, or file hashes have interacted with their internal network.

The Inefficiency: Manual cross-referencing using spreadsheet software is slow, highly susceptible to human error, and delays the Mean Time To Detect (MTTD) a breach.

The Solution: A localized, automated correlation engine that programmatically ingests raw logs, standardizes the formatting, and instantly identifies active network threats.

3. System Architecture & Data Flow

To ensure a frictionless demonstration for portfolio evaluators, the tool was architected as a serverless, client-side application. The data flow follows a strict pipeline:

Data Ingestion Layer: The UI accepts raw text inputs via HTML textareas, simulating file ingestion for .txt and .csv formats.

Sanitization Module: The raw strings are split into arrays by line breaks. Comments (lines beginning with #) and empty whitespaces are programmatically stripped to prevent parsing errors.

Extraction Module: The engine isolates the specific Indicators of Compromise (IOCs) from the structured CSV firewall logs (extracting Column 2 while ignoring timestamps and severity labels).

Correlation Engine: The sanitized arrays are converted into mathematical Sets. The engine filters the data to find exact intersections between the external intelligence and internal logs.

Validation Layer: Overlapping data is passed through Regular Expression (Regex) validators to accurately categorize the threat type before output.

```
 [ Feed A: OSINT Watchlist (.txt) ]         [ Feed B: Firewall Logs (.csv) ]
                 |                                          |
                 v                                          v
[ Sanitize: Strip Comments & Whitespace ]     [ Extract: Isolate IOC Column ]
                 |                                          |
                 +--------------------+---------------------+
                                      |
                                      v
                  [ CORRELATION ENGINE (Set Intersection) ]
                  [   Cross-reference Feed A vs Feed B    ]
                                      |
                                      v
                           < Is there a match? >
                            /                 \
                     [ NO ]              [ YES ]
                       |                    |
                  (Discard)                 v
                              [ Validation & Categorization ]
                              [  Regex: IP, Hash, or Domain ]
                                            |
                                            v
                            [ Output: high_risk_blocklist.txt ]
                            [    Trigger "HIGH RISK" Alert    ]
```

Figure 1: System Architecture & Data Flow.

This diagram illustrates the complete correlation pipeline, from the ingestion and sanitization of raw OSINT and firewall logs to the mathematical set-intersection logic and final generation of the actionable blocklist.

4. Technical Specifications & Algorithms

4.1 Technologies Used

Frontend UI: HTML5, CSS3 (Grid layout for responsive design)

Core Logic: Vanilla JavaScript (ES6+)

Deployment: GitHub Pages (Static Hosting)

4.2 Algorithmic Efficiency (Time Complexity)

A naive approach to comparing two lists involves nested loops, resulting in an O(N×M) time complexity, which would crash a browser if evaluating millions of firewall logs. To ensure enterprise-grade performance, this tool utilizes JavaScript Set objects. By converting the internal firewall logs into a Hash Set, the correlation engine achieves an O(1) constant lookup time. This reduces the overall correlation time complexity to O(N), allowing the tool to process thousands of indicators near-instantly without server-side processing.

4.3 Data Validation (Regex)

To ensure the output is actionable for a firewall administrator, the engine dynamically categorizes the correlated threats using strict Regex patterns:

IPv4 Validation: /^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(...)$/ ensures only mathematically valid IP addresses are tagged as such.

SHA-256 Validation: /^[a-fA-F0-9]{64}$/ ensures exactly 64-character hexadecimal strings are properly tagged as file hashes.

Fallback: Any overlapping indicator failing those two strict checks is classified as a Domain/URL.

5. Security & Privacy Advantages

By engineering this as a strictly client-side application, the tool offers a massive operational security (OpSec) advantage. Internal firewall logs often contain highly sensitive corporate data. Because the JavaScript runs entirely within the user's local web browser, no firewall logs are ever transmitted over the internet or uploaded to a third-party server. Analysts can safely paste proprietary logs into the sandbox with zero risk of a data leak.

6. Simulation Results & Output Data

To validate the correlation engine, a simulation was run utilizing two pre-loaded data sets: an OSINT Watchlist (Feed A) containing 4 known malicious indicators, and internal Firewall Logs (Feed B) containing 4 network connection events.

Upon execution, the engine successfully stripped the metadata, correlated the arrays, and accurately identified two active threats that were present in both feeds.

Raw Correlation Engine Output:

[!] HIGH RISK - Repeated IOC Found: 10.0.0.55 (IP Address)
[!] HIGH RISK - Repeated IOC Found: malicious-domain.com (Domain/URL)

System Action Taken:
Following the successful correlation, the system successfully categorized the data types using Regex and virtually generated a unified high_risk_blocklist.txt file ready for firewall deployment.

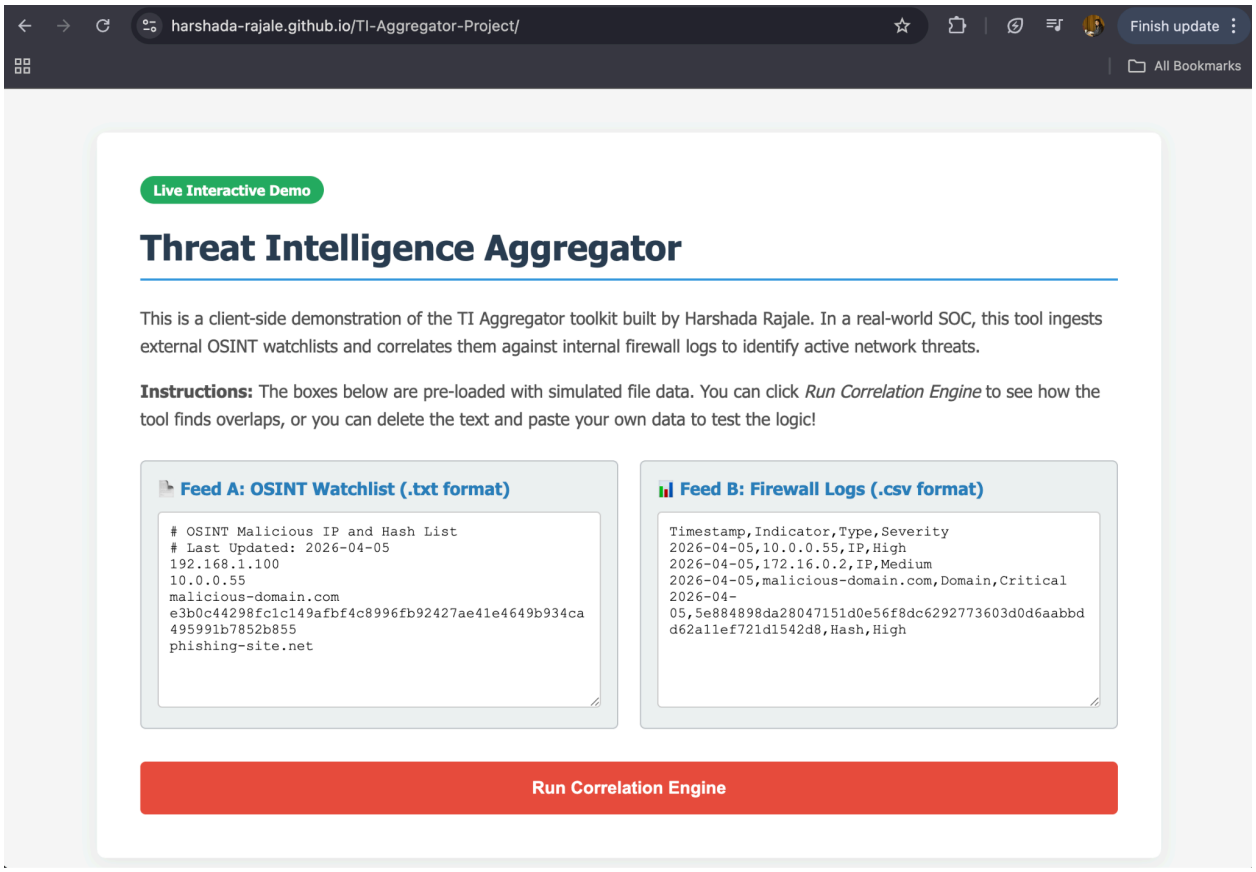

Figure 2: The Input Interface

Figure 1: The interactive web interface of the Threat Intelligence Aggregator, displaying the simulated ingestion of an external OSINT watchlist (Feed A) and internal firewall logs (Feed B) prior to correlation.

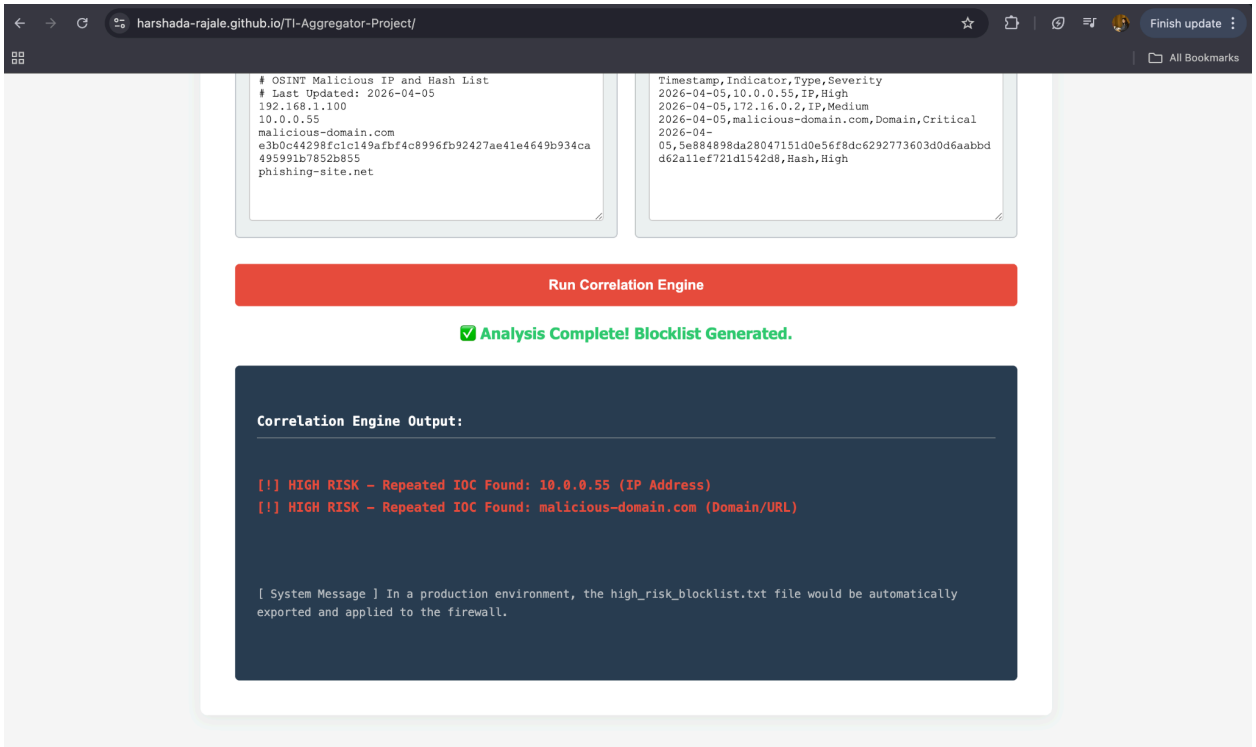

Figure 3: Correlation Engine output

The Correlation Engine output successfully identifying and categorizing active network threats based on set-intersection logic, generating a unified master blocklist.

7. Conclusion

The Threat Intelligence Aggregator successfully automates one of the most tedious and critical tasks in a SOC. It demonstrates a deep understanding of data structures, efficient algorithms, format sanitization, and secure application architecture. The deployed interactive web application serves as a robust proof-of-concept, validating that the underlying logic is highly accurate and ready for integration into a broader cybersecurity defense pipeline.